ISHTIRAK

Distributed Neighborhood Capacity Coordination

Final project report | Distributed Systems | 29 October 2026

A finite generator budget, coordinated through a majority-committed ledger.

Three Go coordinators agree on capacity grants. Building gateways aggregate MQTT readings and enforce short-lived budgets in a simulated served-load calculation. PostgreSQL preserves audit and telemetry, and a React dashboard exposes normal operation and controlled failures.

120 A capacity. Three coordinators. Eleven integration phases passed.

The measured run admitted 17 of 40 competing 7 A requests, reserving 119 A. No capacity violations were observed in 708 coordinator samples, nor in a randomized fault check of 1,529 requests. The report distinguishes this evidence from a formal proof and explains the timing assumptions and prototype limits.

Application: http://localhost:8088

Source, reproduction steps, raw measurements, and an editable PowerPoint accompany this report.

1. Problem and outcome

A neighborhood generator has a finite amperage budget, while buildings independently observe changing household demand. If buildings make allocation decisions independently, concurrent requests or retries can reserve more than the generator can supply. A coordinator crash introduces a second problem: a replacement must distinguish capacity that is still authorized from capacity that can safely be reused.

Ishtirak addresses this coordination problem with a replicated capacity ledger and short-lived building leases. The prototype uses Go meter simulators, MQTT/Mosquitto, three Go building gateways, three HashiCorp Raft coordinators, PostgreSQL, and a React dashboard. Its central invariant is:

Total granted integer amperage <= committed generator capacity.

The supplied demonstration uses a 120 A generator and initially 50 simulated meters across Hamra, Mar Mikhael, and Achrafieh. A gateway computes simulated served demand as the smaller of observed demand and its currently valid lease budget. No physical breaker, relay, or electrical installation is controlled.

The implemented application, evaluation scripts, report, and PowerPoint cover the deliverable categories in the supplied project brief. The original brief, proposal, and flow diagram remain unchanged. Assessment of the final grade remains with the instructor.

2. Why this is a distributed system

The system has autonomous processes with separate memory and explicit network communication. Each coordinator has its own persistent Raft log and snapshot volume. There is no shared-memory capacity map between coordinators. MQTT messages connect meter producers to building consumers; HTTP connects gateways to the coordinator service; Raft replicates ordered commands over TCP; SQL stores a recoverable projection; HTTP and WebSocket expose observations to the dashboard.

Component	Partition or responsibility	Communication and state
Meter simulator	Independent logical household meters; 10/50/200 evaluated	MQTT QoS 1; unique reading IDs
Mosquitto	Decouples producers from building subscribers	Topic-based publish/subscribe
Three gateways	One building per process; aggregate its meter readings	MQTT input, HTTP grants/readings; local lease deadlines
Three coordinators	Replicated authority over the shared generator budget	Raft TCP; HTTP/JSON and WebSocket; three separate volumes
PostgreSQL	Durable audit and reading projection	SQL; single database, no grant authority
React + local controller	Operator visibility and bounded fault demonstrations	HTTP/WebSocket; label-checked Docker actions

The brief gives "Database, Application and Logging service" as example components. In Ishtirak the **database** is PostgreSQL, the **application** is the meter simulators, gateways, coordinators and dashboard, and the **logging service** is the replicated audit log: every committed grant or reclamation becomes an event in the Raft state machine's outbox, is projected by the observed leader into an append-only SQL audit table with deterministic event IDs, and is shown and exportable in the dashboard's Activity log. There is no separate logging process; logging is a responsibility of the coordinators and PostgreSQL, and it keeps catching up if PostgreSQL is unavailable.

There are eleven Compose services. Meter identities are multiplexed by one simulator process rather than launching 200 containers. The dashboard and controller are distinct services. Compose runs these processes on one host: this demonstrates distributed protocols and process/network failures, but does not tolerate loss of that host. The local fault controller is demonstration infrastructure, not an additional source of allocation authority.

Communication path

1. Meters publish identified readings to building-specific MQTT topics.

2. A building gateway deduplicates readings, removes stale demand samples, and aggregates demand.

3. The gateway asks a coordinator for a building lease. Followers redirect it to the known leader; redirects are restricted to configured coordinator endpoints.

4. The leader appends a command to Raft and waits for its committed state-machine result.

5. The gateway accepts only a matching, timely reply and applies its safe local deadline.

6. Audit events are projected from committed Raft state into SQL. Reading batches are acknowledged only after durable SQL insertion.

7. The UI polls combined observations and consumes live coordinator WebSocket updates. Its status values are observations, not a linearizable read of the entire distributed system.

The implementation separates telemetry partitioning by building from replication of the single shared capacity budget. Adding a building partition can scale telemetry handling, but every allocation still passes through one logical consensus ledger.

3. Deterministic consistency and concurrency

The ledger starts uninitialized. A replicated init_capacity command establishes capacity once; later nodes replay the same value instead of deriving it from local configuration. Grants fail closed before initialization. Incompatible legacy histories fail closed rather than being reinterpreted as safe capacity state.

Each grant request contains request_id, building_id, session_id, and positive integer amps. The state machine stores a canonical payload hash with the result. Repeating the same request returns the stored result; changing the payload under that ID produces request_id_conflict. A timeout is an unknown outcome, so a gateway retries the original ID and retains its original send time.

Raft serializes commands, and the ledger protects state access with a mutex. Admission computes the incremental reservation and compares it with available capacity. A renewal for the same session cannot shrink an existing reservation. A different session cannot replace a currently reserved building lease.

Each successful grant version is its unique committed log index.

An induction argument explains the accounting invariant. Initialization creates an empty ledger with positive capacity. A grant increases reservations only if its increment fits the available capacity. Rejected commands and duplicate requests do not increase reservations. Expiry only removes the matching reservation. Consequently, each committed transition preserves the capacity bound. This argument depends on the implementation matching the transition rules; unit tests, review, race checks, and live stress supply evidence for that correspondence.

Snapshot restoration includes initialized capacity, leases, request hashes and outcomes, the audit counter, and the event outbox. Restore validation rejects invalid or over-capacity state. It does not discard idempotency history merely because a log has been compacted.

4. Leases, failures, and timing assumptions

Lease safety is intentionally conservative. The protocol has a fixed 12-second lease duration, enforced at coordinator startup. A gateway may use an accepted grant only until its original first-send monotonic time plus 10 seconds. The two-second margin is subtracted locally. Retries and duplicate replies never move that original send time forward. Delayed, expired, or identity-mismatched replies are rejected.

The coordinator holds a newly observed lease version for 12 monotonic seconds before proposing expiry. A leadership-term change restarts inherited holds. Expiry is a replicated command fenced by the building ID, expected lease version, and proposing Raft term. A stale leader or stale expiry cannot erase a newer version. The ledger itself never consults a wall clock to reclaim capacity.

With a relative monotonic clock-rate error bound rho, the gateway's latest stop must precede the coordinator's earliest expiry: 10/(1-rho) < 12/(1+rho). The documented operational assumption is at most 1% rate error. At that bound, the respective durations are about 10.10 and 11.88 real seconds; the coordinator also starts its hold no earlier than observation of the committed lease. Absolute wall-clock offsets do not decide safety. Host suspend behavior, malicious processes, and physical actuation delays are outside the certification of this simulator.

Event	Designed response	Trade-off
One coordinator fails	Two remaining nodes elect a leader and commit	Temporary interruption during election
Majority is unavailable	No new grant can commit; local service deadlines expire	Availability is sacrificed to prevent conflicting grants
Gateway crashes	Its reservation remains until committed expiry; restart has a fresh session	Reconnection can wait for conservative reclamation
A response is delayed or duplicated	Original send time and result identity remain binding	A valid committed grant may arrive too late to use
PostgreSQL fails	Raft grants continue; audit outbox catches up after recovery	Reads/history are degraded; bounded gateway queues may lose data
Broker or gateway is unavailable	Telemetry becomes missing/stale; current lease rules still apply	No broker or gateway replication

Renewals retain the larger reservation until expiry. This avoids releasing capacity while an older, larger authorization could still be valid, but it can waste capacity after demand falls. Fair scheduling and starvation prevention are not implemented. The system demonstrates safety under constrained availability; it does not promise optimal utilization.

This trade-off was observed live. After the 200-meter load run, one building held a reservation of 107 to 111 A while the other two showed expired leases and 0 A served, although total demand in the live observation was only about 75 A (the dashboard figures are not in the JSON evidence): a renewal never shrinks a reservation while the same gateway session keeps renewing, and the remaining capacity was smaller than either other building's request. Total grants stayed within 120 A, so the invariant held; utilization and fairness did not. For demonstrations, python scripts/reset_demo.py restarts the three gateways (fresh sessions), lets the old reservations expire through the normal committed expiry, and returns the buildings to their current demand (in one observed run 24, 26 and 27 A, 77 A in total). A production design would need a safe shrink rule that waits until every older, larger grant at the gateway has expired; that change touches lease safety and was deliberately not made here.

5. Persistence and error handling

Three named volumes retain the coordinators' logs and snapshots. PostgreSQL has its own volume. Deterministic audit event IDs and SQL primary keys make projection retries idempotent. The observed leader exports committed audit events; the FSM retains the outbox for catch-up. Reading IDs similarly prevent duplicate SQL rows.

MQTT QoS 1 can deliver duplicates. Gateways deduplicate readings and use bounded queues; pending and dropped counts are visible. Failed SQL flushes retain their batch until a later retry. Readings during gateway downtime may be lost, and a gateway crash loses its in-memory buffer. These limitations match the proposal's unreplicated gateway and single-database scope.

HTTP handlers validate and bound input sizes. The local control service accepts only named actions for this project's containers and checks their Compose labels before Docker operations. The control service rejects cross-origin browser requests, and the dashboard's proxy accepts only localhost and 127.0.0.1 Host values (HTTP 421 otherwise, which closes DNS-rebinding hostnames) and refuses browser-originated writes to the coordinators (HTTP 403; local scripts send no Origin header and are unaffected). There is still no authentication on the coordinator or control APIs. The dashboard is bound to 127.0.0.1:8088. The controller's Docker socket and the public demo database password make this a trusted local demonstration, not a public deployment configuration.

Raft request history and the audit outbox are retained without an unbounded-production retention policy. This improves prototype replay clarity but creates memory/snapshot growth. Resetting Raft history while preserving old SQL event IDs can invalidate projection correspondence; only an intentional, coordinated reset of the complete stopped demo is appropriate.

6. Measured evaluation

The live run on 29 October 2026 is stored in artifacts/evaluation.json, run ID f85446506835. It ran from 11:25:41 to 11:31:09 UTC against the actual Docker stack. The runner uses Python's standard library, bounded timeouts, and a finally restoration path. All eleven phases passed, all controlled services were restored, and the meter configuration returned to 50 meters with demand factor 1. Latency percentiles use linear interpolation between order statistics. Gateway-crash, leader-failure, load and end-to-end measurements are repeated (3, 3, 5 and 5 trials); the tables give the mean, the sample standard deviation (sd) and the range, and evaluation.json keeps every trial.

Capacity and fault results

Scenario	Measured result
Real MQTT and SQL readiness	506,853 durable readings already present at readiness
Gateway crash and reclamation (3 trials)	Lease reclaimed 11.97, 10.80 and 14.25 s after the crash (mean 12.34, sd 1.76); each restart used a fresh session. The crashed building was whichever one held a lease
Leader failure (3 trials)	Three different leaders in turn (coordinator2, 3, 1 stopped); a replacement accepted a grant after 4.05, 4.06 and 6.34 s (mean 4.82, sd 1.32)
Majority loss	Three new requests returned HTTP 503; simulated service was 0 A after 13 s
PostgreSQL outage	Grant accepted while SQL was down; its exact audit request appeared once after recovery; audit rows rose from 8,291 to 8,314; 550 additional durable readings; 66 readings pending during the outage
Concurrent capacity stress	Forty requests of 7 A: 17 accepted, 23 rejected, 119 A committed under 120 A
Duplicate and conflict	Duplicate preserved the lease; changed payload under the same ID rejected

The leader-failure measurement includes routing and a successful grant, not just election time. Gateway reclamation is an observation made through the orchestration path, not an exact internal timer measurement. A 503 during majority loss correctly expresses unavailable/unknown commit outcome, not a definitive rejection that may safely be retried under a new ID.

The sampler collected 243 combined overviews and 708 coordinator snapshots, with zero transport errors, a maximum observed grant of 119 A, and no sampled invariant violations. It also checked gateway served demand against local valid budgets. Polling cannot prove that no transient existed between observations; deterministic state-machine checks and timing design supply complementary evidence.

Randomized fault check

scripts/chaos_check.py is a small history checker in the spirit of Jepsen. With a fixed seed it builds a deterministic fault schedule, runs eight concurrent workers that send randomized grants, duplicate replays and one changed-payload replay to random coordinators while the schedule stops and restarts services, samples the capacity ledger about once a second, then heals everything and checks the recorded history. Run e2ec4860 (seed 1, 90 s, 11:13:35 to 11:15:06 UTC) stopped and restarted coordinator2, the Hamra and Mar Mikhael gateways, and included a majority-loss window in which coordinator3 and coordinator1 were down together for about 15 s.

Check	Result
Requests sent	1,529: 284 accepted, 1,183 rejected, 62 unknown outcomes
Rejections	1,182 capacity_exhausted; 1 request_id_conflict (the deliberate changed-payload replay)
Unknown outcomes	HTTP 503 or timeout, counted separately because they may have committed
Capacity invariant	134 samples; maximum observed grant 120 A (the limit, not above it); 0 violations
Unique versions	No committed version was returned for two different request IDs
Recovery	All seven faultable services restarted; the three coordinators reported the same applied index (22,808)

The synthetic chaos-* buildings share the 120 A budget with the real gateways, which is why capacity rejections dominate. Like the evaluation sampler, this is observational evidence from one seed, not a proof; the unit tests show the checker flags an over-capacity history and duplicate versions.

Performance and scaling

Each load settled for 10 seconds, then ran five trials of 200 allocation requests with four concurrent HTTP clients, using a new synthetic benchmark building and session per trial and unique request IDs, alongside the real building gateways. All 200 decisions in every trial were accepted; the separate capacity stress tested denials. No benchmark had a transport/HTTP error. Values are mean ± sd over the five trials (range in brackets).

Active meters	p50 ms	p95 ms	Decisions/s
10	79.5 ± 1.0 (78.2-80.7)	113.3 ± 4.4 (109.5-120.1)	49.4 ± 0.4
50	78.3 ± 0.6 (77.7-79.2)	112.9 ± 2.3 (110.9-116.5)	50.0 ± 0.6
200	79.4 ± 1.4 (77.1-80.6)	114.1 ± 3.9 (109.1-119.2)	48.6 ± 0.3

Observed gateway MQTT readings per second in the median trial's window were 9.7, 57.8 and 193.9 for 10, 50 and 200 meters, and SQL insert deltas were 53, 283 and 1,066 readings. Batching and observation boundaries explain why these differ from the nominal rates.

These are HTTP allocation-decision latencies, including local proxy/network and commit time. The decision benchmark does not exercise meter volume, so its flat latency across meter counts says little about meter scalability by itself; the meter and reading volume is shown by the readings per second above.

Concurrency sweep. At 50 meters the same 200-decision method was run with 1, 4, 16 and 64 concurrent clients:

Clients	p50 ms	p95 ms	Decisions/s
1	38.8	54.8	25.9
4	82.1	125.2	46.8
16	275.6	437.0	52.7
64	300.6	517.3	51.6

There were no transport errors. Throughput rises with load and then plateaus near 50 decisions/s while latency grows, which is the shape expected when requests queue behind one serialized resource. Every grant is ordered by the single Raft leader, so this is consistent with the commit path being the limit, but the cause was not profiled: the dashboard proxy and the Python client are also in the path. The plateau is therefore a measured ceiling for this setup, not an identified bottleneck.

End-to-end telemetry latency. Stepping the demand multiplier from 0.5 to 1.0 at 50 meters, the change became visible in the gateways' observed demand 0.38 to 0.83 s after the request was sent (mean 0.67 s, sd 0.18 s, five samples; polling every 0.1 s, largest poll gap 166 ms). This covers the control request, meter publish, MQTT, gateway aggregation and dashboard refresh, and is an upper bound at the polling resolution. Served demand moved at the same instant in each sample because a gateway computes it from the same snapshot (the smaller of demand and its lease budget); when a lease budget is the limit, served demand stays flat while demand rises.

One host, one run per configuration, short windows, concurrent local activity, and no network-delay injection limit these conclusions, and the trials are repeats on one stack rather than independent samples. Further evaluation should use dedicated hosts, profiling of the commit path, longer soak tests, and deliberate packet loss and partition schedules. There is no measured claim of unlimited scale.

Software and browser checks

The integrated Go code passed go test -race ./... and go vet ./... in golang:1.25. There are 23 named test functions covering ledger concurrency, duplicate/conflict handling, version/term expiry fences, snapshot restoration, capacity initialization, fixed TTL, inherited deadlines, late replies, gateway restart identities, bounded reading queues, redirects, simulator configuration, and local control restrictions. The Python tooling has 35 unit tests (21 for the fault checker, 14 for the statistics helpers) and the dashboard's scenario logic has 18 Vitest tests.

The TypeScript/Vite production build passed. npm audit --omit=dev reported zero known vulnerabilities for the final dependency set (React 19, Recharts, Motion, Radix Dialog, Inter). This is a dependency-database result, not a comprehensive security certification.

Seventeen automated Chrome checks passed: three buildings and coordinators, no desktop overflow, live WebSocket frames, dialog focus, Escape handling, real simulator change to 10 meters and back to 50, populated audit events, the Resilience lab rendering, a real leader-failover scenario run from the UI with every coordinator restored, the capacity proof and observed timeline, no mobile overflow, mobile navigation, and no uncaught JavaScript errors. Desktop, lab and mobile screenshots were visually reviewed. Evidence is in artifacts/browser-check.json and the accompanying PNG files.

Dashboard and Resilience lab

The dashboard is a React 19 single-page application: live capacity metrics, a Recharts telemetry chart built only from real observations (no synthetic history), building cards with lease timers, the coordinator cluster, a searchable audit log with JSON export, and a Simulation lab for meter count and demand. The Resilience lab runs four guided failure scenarios (leader failover, majority loss, gateway crash, database outage) through the label-checked controller. Each scenario restores every stopped service on completion, timeout, cancel or navigation, and the lab shows a capacity proof (peak granted against 120 A, violations, fresh samples), a per-coordinator Raft view (term, applied index, last log index) and a timeline of observed leader and availability changes. Every value is an observation from dashboard polling, labelled as such, and stale samples never count as proof.

7. Requirement and rubric mapping

Brief requirement / criterion	Concrete implementation or evidence
Clear, significant problem	Concurrent building demand against one finite generator budget; Sections 1 and 3
At least three components (brief's example: database, application, logging service)	Meter/broker, gateways, three coordinators, PostgreSQL (database), replicated audit log projected to SQL (logging service), UI/controller; Section 2 and Compose
Data partitioning	MQTT topics and one gateway per building
Communication mechanisms	MQTT QoS 1, HTTP/JSON, WebSocket, Raft TCP, SQL
Concurrency and consistency	Ordered majority-committed FSM, idempotency, atomic capacity checks; randomized fault check of the recorded history
Fault tolerance	Leader replacement, majority-loss refusal, safe lease expiry, SQL catch-up; repeated failover and reclamation trials; randomized fault schedule with a majority-loss window
Performance and scalability evaluation	10/50/200 meter results over five trials with mean, sd and range; concurrency sweep to 64 clients; end-to-end telemetry latency; explicit methodological limits
Challenges and trade-offs	Unknown outcomes, duplicates, stale leaders, monotonic timing, safety versus availability, first-come allocation without shrinking, and retention; Sections 3-6
Creativity and practical relevance	Neighborhood-specific capacity coordination with a live operator, a Resilience lab that breaks the running system on purpose, and a capacity proof drawn from observations
Functional application	docker compose up --build -d; local dashboard at port 8088
Documentation	This report, README, source tests, measured JSON evidence
PowerPoint presentation	docs/Ishtirak_Presentation.pptx, with architecture, why it is distributed, safety, the live dashboard and lab, results, trade-offs and limits, and speaker notes

The project addresses each required deliverable and the named distributed-systems concepts. This mapping is evidence of coverage, not a promise of a particular rubric score.

8. Reproduction and demonstration

Start Docker Desktop with Linux containers. In the project root run docker compose up --build -d, then open http://localhost:8088. Internet access is needed for the first image/dependency build. No host Go compiler or external AI credential is required.

Run python scripts/evaluate.py to repeat the fault and load sequence (about 5 to 12 minutes; the recorded run took 5.5 minutes, and a failing run is bounded by phase deadlines). Run python scripts/chaos_check.py --seed 1 --duration 90 for the randomized fault check; the same seed reproduces the same fault schedule, and --dry-run prints it without touching the stack. Run them on a demo you are prepared to interrupt: they temporarily stop only this project's named services, preserve volumes, and

restore services and the prior meter configuration. Do not change meter counts in the UI while they are measuring. After a 200-meter run, python scripts/reset_demo.py returns the buildings to a fair allocation (see Section 4). Python unit tests: python -I -m unittest discover -s scripts -p "test_*.py"; dashboard tests: npm test in web.

Run node scripts/browser-check.mjs for browser checks. This uses Node 24 and local Chrome; set CHROME_PATH if Chrome is installed elsewhere. It creates a dedicated temporary browser profile under artifacts, changes the simulator to 10 and back to 50, and closes the test browser. It does not use the user's normal browser profile.

For a live presentation, open the Resilience lab and run the scenarios in this order, letting each recover before the next: leader failover (the observed timeline shows the leader lost and a new one observed while the capacity proof stays at zero violations), majority loss (new grants stop and service reaches 0 A), gateway crash (conservative reclamation), then database outage (catch-up). Then compare 10/50/200 meters in the Simulation lab, and run python scripts/reset_demo.py afterwards. The dashboard distinguishes requested demand, served demand, and reserved capacity.

Use docker compose stop and docker compose start to preserve the demo between sessions. Do not remove volumes unless intentionally resetting all prototype data. The project does not publish or deploy to an external service.

9. Engineering review and remaining limits

Implementation used parallel workers in isolated copies for coordinator and telemetry changes; the main author integrated their bounded paths. An independent reviewer challenged the lease design and checked safety corrections, including replicated capacity initialization and a fixed lease protocol. Deterministic tests cover these changes.

The later additions (the redesigned dashboard, the Resilience lab, the randomized fault checker and the repeated-trial evaluation) did not modify consensus, ledger or lease code. They were built by isolated workers, integrated by the main author and checked by running the project's own tests and the live runs described in Section 6; running them exposed and fixed three defects in the new evaluation code itself (a probe that reused a still-leased building, a timing signal capped by the lease budget, and a trial that assumed a fixed building held the lease). An independent read-only audit then compared the deliverables with the brief, its rubric and every figure in this report; its confirmed findings were fixed. No external review of the safety design has been performed, and the independent reviews do not amount to a formal verification.

Known design limitations remain: no physical electrical certification; no host, broker, database or gateway failover; potentially lost readings during outages; histories grow over time; first-come allocation with no fairness guarantee, so a building that once reserved a large amount can starve the others until its gateway restarts; no active reservation shrinking; a measured decision-throughput ceiling of about 50 per second whose bottleneck was not profiled; observed UI state may lag; and no production authentication/TLS design. Optional image/TypeSafe ideas were not added to authoritative allocation or to the prototype's scope.

The next engineering steps, if scope is expanded, are repeated multi-host testing, clock/suspension experiments, bounded history retention with replay guarantees, fair admission policies, and a separate authenticated deployment design. These are future work, not implemented capabilities.

10. Sources and evidence index

• Final-Project 2025.pdf: course brief, required deliverables, evaluation rubric.

• Ishtirak_Project_Proposal_slim.docx: proposed architecture, invariant, experiments, limited scope.

• Ishtirak_Project_Flow.drawio: original project flow.

• HashiCorp Raft API: https://pkg.go.dev/github.com/hashicorp/raft

• Eclipse Paho Go MQTT API: https://pkg.go.dev/github.com/eclipse/paho.mqtt.golang

• internal/ledger/ledger.go and its tests: authoritative transitions and replay.

• internal/coordinator/server.go, deadlines.go and tests: Raft lifecycle, grants and expiry.

• internal/telemetry/gateway.go and tests: safe local use, retries and buffering.

• scripts/evaluate.py, artifacts/evaluation.json: measured fault/load results, repeated trials, concurrency sweep, end-to-end latency and raw response timings.

• scripts/chaos_check.py, artifacts/chaos-check.json: seeded randomized fault schedule, recorded history and invariant check.

• scripts/reset_demo.py: returns the demo to a fair allocation after a demand spike.

• web/src/resilience-lab.tsx, web/src/lab/scenarios.ts: guided failure scenarios, capacity proof and observed timeline.

• scripts/browser-check.mjs, artifacts/browser-check.json: browser assertions and screenshots.

• README.md, compose.yaml: startup, operation, configuration and limits.